

UNIDAD 5

IMPLEMENTATION DE JWT

Ejercicio: Sistema de gestión de biblioteca digital

Diseña e implementa el módulo de autenticación y autorización para una API REST de una biblioteca digital utilizando Spring Boot + Spring Security + JWT.

Por ahora, el sistema debe trabajar sin repositorios, sin entidades y sin base de datos. Los usuarios pueden almacenarse temporalmente en memoria.

Como base se utilizará el Proyecto publicado en:

<https://github.com/sistemasfm/ejercicio8.git>

Requisitos

- Roles disponibles: ROLE_LECTOR, ROLE_BIBLIOTECARIO, ROLE_ADMIN.
- POST /api/auth/register → cualquiera puede registrarse. El rol asignado por defecto será ROLE_LECTOR.
- POST /api/auth/login → endpoint público. Devuelve un JWT con expiración de 1 hora.
- GET /api/libros → puede ser consultado por cualquier usuario autenticado.
- POST /api/libros → solo BIBLIOTECARIO y ADMIN pueden cargar libros.
- DELETE /api/libros/{id} → solo ADMIN puede eliminar libros.
- POST /api/prestamos → solo LECTOR puede solicitar el préstamo de un libro.
- GET /api/prestamos/mis-prestamos → solo LECTOR puede consultar sus propios préstamos.
- GET /api/prestamos → solo BIBLIOTECARIO y ADMIN pueden ver todos los préstamos.
- PUT /api/prestamos/{id}/aprobar → solo BIBLIOTECARIO puede aprobar préstamos.

Tabla de endpoints esperados

Método	URL	Respuesta esperada	Autorización	Ubicación recomendada
POST	/api/auth/register	201	Público	Filter Chain
POST	/api/auth/login	200	Público	Filter Chain
GET	/api/libros	200 / 401	Usuario autenticado	Filter Chain
POST	/api/libros	201 / 401 / 403	ROLE_BIBLIOTECARIO o ROLE_ADMIN	Filter Chain

DELETE	/api/libros/{id}	200 / 401 / 403	Solo ROLE_ADMIN	Filter Chain
POST	/api/prestamos	201 / 401 / 403	Solo ROLE_LECTOR	Filter Chain
GET	/api/prestamos/mis-prestamos	200 / 401 / 403	Solo ROLE_LECTOR	@PreAuthorize
GET	/api/prestamos	200 / 401 / 403	ROLE_BIBLIOTECARIO o ROLE_ADMIN	Filter Chain
PUT	/api/prestamos/{id}/aprobar	200 / 401 / 403	Solo ROLE_BIBLIOTECARIO	@PreAuthorize

Objetivos del ejercicio

- Registro y login de usuarios.
- Generación y validación de JWT.
- Protección de rutas según rol.
- Uso de SecurityFilterChain y @PreAuthorize.
- Manejo correcto de respuestas 401 y 403.
- Trabajo sin persistencia utilizando estructuras en memoria.